



Manipulaciones geométricas

Métodos constructivos

GeoPandas cuenta con todas las herramientas para trabajar con manipulaciones geométricas contenidas en la librería Shapely. Más adelante, se explicarán otro tipo de operaciones para crear nuevas formas geométricas a partir de diferentes *datasets* espaciales, por ejemplo, las operaciones de **intersección, unión, diferencia**.

En esta sección revisaremos algunos de los métodos constructivos más utilizados con los que podemos trabajar.

■ `GeoSeries.buffer(distance, resolution=16)`

Este método nos devuelve un `GeoSeries` de geometrías que **representan todos los puntos dentro de una cierta distancia de cada objeto geométrico**. Para entender mejor cómo funciona `buffer`, veamos un ejemplo.

Definamos tres elementos gráficos básicos: un punto, una `LineString` y un polígono.

```
from shapely.geometry import Point, LineString, Polygon
s = geopandas.GeoSeries(
    [
        Point(0, 0),
        LineString([(1, -1), (1, 0), (2, 0), (2, 1)]),
        Polygon([(3, -1), (4, 0), (3, 1)]),
    ]
)
```

Al graficar esto, obtenemos el siguiente resultado:

```
s.plot()
```

<matplotlib.axes._subplots.AxesSubplot at 0x7fa06ba81610>

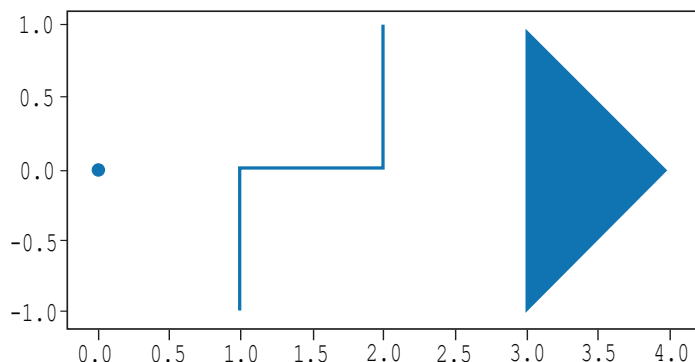


Figura 1. Elementos gráficos básicos.

Como puedes observar, tenemos un punto, una `LineString` y un polígono.

Ahora, si aplicamos la función `buffer` con un parámetro de 0.2 a estas geometrías, obtendremos el siguiente resultado:

```
s.buffer(0.2).plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fa06bed7350>
```

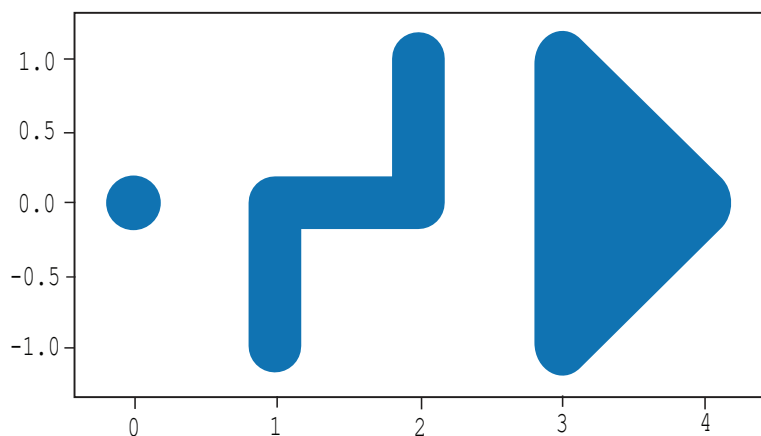


Figura 2. Resultado de un `buffer` de 0.2 unidades que “rodea” a las geometrías originales.

Además de la distancia, como parámetro también podemos utilizar `cap_style` y `join_style`, observa el ejemplo.

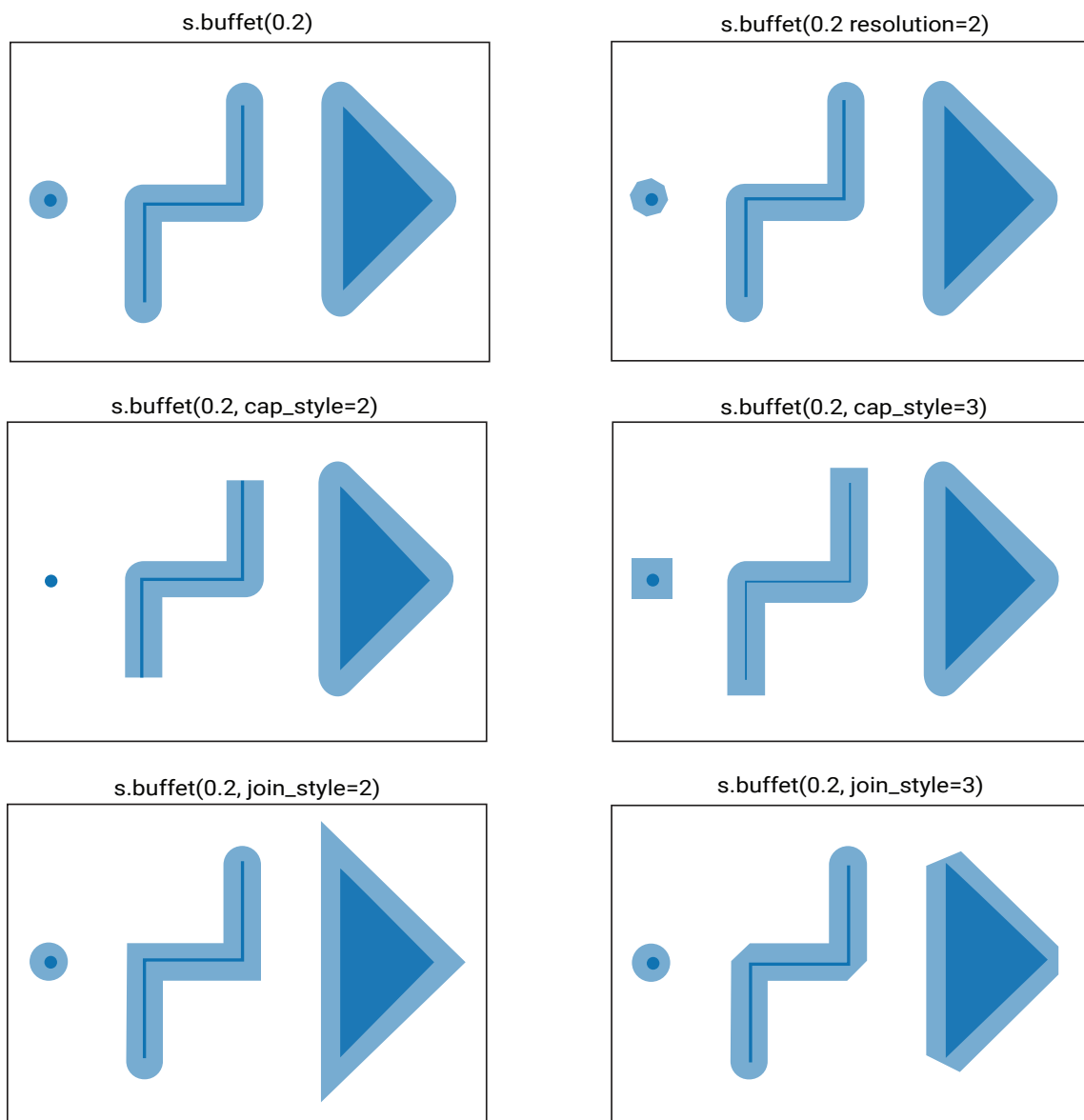


Figura 3. Parámetros `cap_style` y `join_style`.

GeoSeries.boundary

Este método nos regresa un **GeoSeries de objetos de una dimensión inferior que representan los límites de cada geometría.**

Continuando con el ejemplo de las tres geometrías definidas previamente, al aplicar la función **boundary**, obtenemos el siguiente resultado:

```
s.boundary
```

```
0          GEOMETRYCOLLECTION EMPTY
1    MULTIPOINT (1.00000 -1.00000, 2.00000 1.00000)
2    LINESTRING (3.00000 -1.00000, 4.00000 0.00000,...
dtype: geometry
```

Figura 4. Función `boundary`

```
s.boundary.plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fa06bda4a10>
```

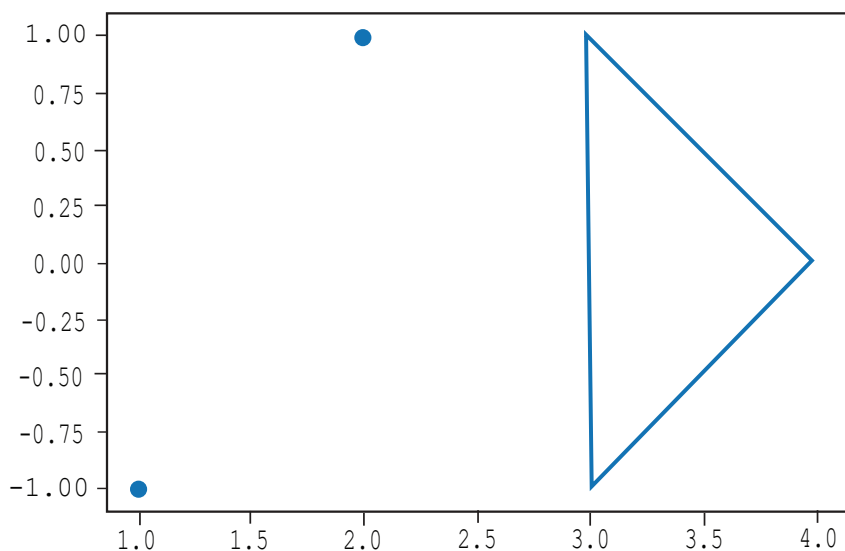


Figura 5. Función `boundary` aplicada a las tres geometrías.

Como podemos observar, el `boundary` del punto nos regresa una `GEOMETRYCOLLECTION EMPTY`, el `boundary` de la `LineString`, nos regresa un `MULTIPOINT (1.00000 -1.00000, 2.00000 1.00000)`, que es precisamente los puntos de inicio y fin de la `LineString` (los límites) y finalmente, el `boundary` del polígono, nos regresa un `LINESTRING (3.00000 -1.00000, 4.00000 0.00000, ...` que es el contorno (el perímetro) del polígono.

■ `GeoSeries.centroid`

El `centroid` o centroide, es método nos devuelve una `GeoSeries` de puntos que representan el centroide de cada geometría. **El centroide de una figura es el punto que representa su centro geométrico.**

Usando el mismo ejemplo con el punto, la `LineString` y el polígono, así quedarían los centroides:

```
s.plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fa06ba81610>
```

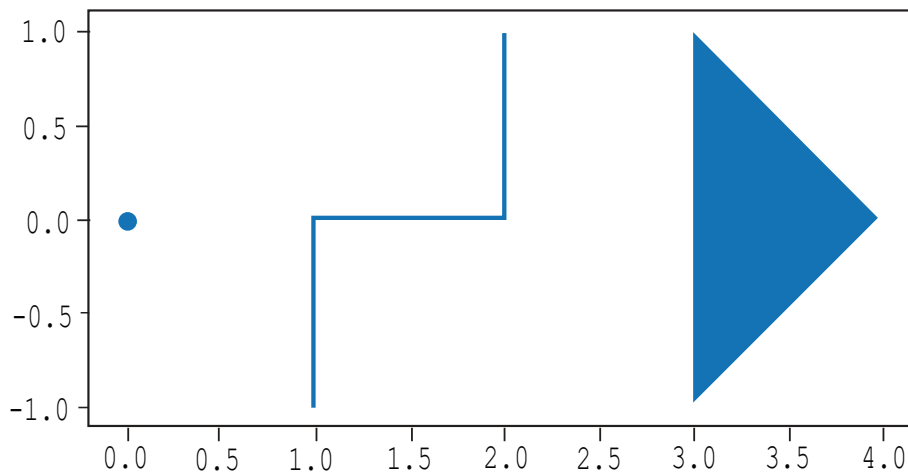


Figura 6. Centroides aplicados al ejemplo de las figuras geométricas.

```
s.centroid
```

```
0    POINT (0.00000 0.00000)
1    POINT (1.50000 0.00000)
2    POINT (3.33333 -0.00000)
dtype: geometry
```

```
s.centroid.plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fa06b8b7410>
```

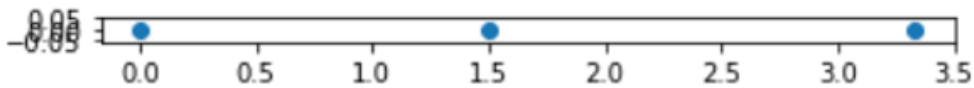


Figura 7. Función `centroid`.

Como podemos ver, el centroide de un punto es el punto mismo, el centroide de la `LineString` está quedando en el punto (1.5, 0) y el centroide del triángulo queda localizado en el punto (3.3333, 0).

GeoSeries.convex_hull

Este método nos regresa una `GeoSeries` de geometrías que representa el polígono convexo más pequeño que contiene todos los puntos en cada objeto, a menos que el número de puntos en el objeto sea menor que 3. Un **polígono convexo**, es un polígono cuyos ángulos internos miden no más de 180 grados. Este polígono es llamado “**casco convexo**” (`convex hull`).

Apliquemos la función `convex hull` al ejemplo de las figuras:

```
s.plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fa06ba81610>
```

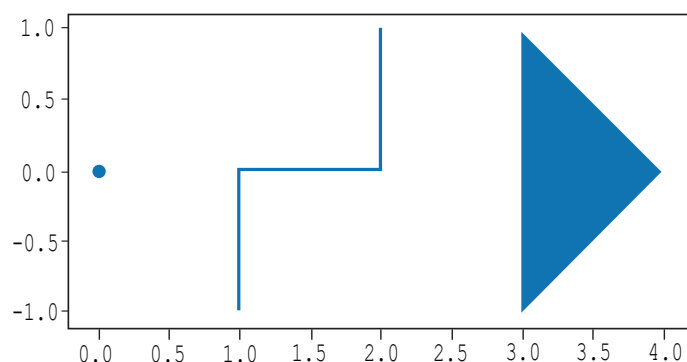


Figura 8. Antes de aplicar la función `convex hull`.

```
s.convex_hull.plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fa06b8bdf10>
```

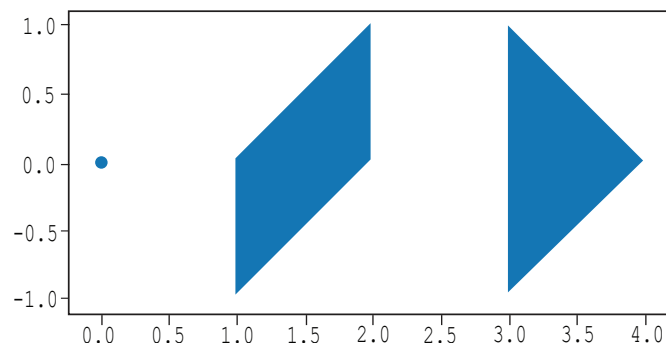


Figura 9. Después de aplicar la función `convex hull`.

En este caso, el casco convexo de un punto es el punto mismo, de la `LineString` es el polígono que podemos ver en la parte de enmedio que cubre completamente a la `LineString`, y finalmente, el del triángulo, es el triángulo mismo.

GeoSeries.envelope

Este método nos regresa una `GeoSeries` de geometrías que **representa el punto o el polígono rectangular más pequeño** (con lados paralelos a los ejes coordenados) que contiene a cada objeto.

```
s.envelope.plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fa06b6ea410>
```

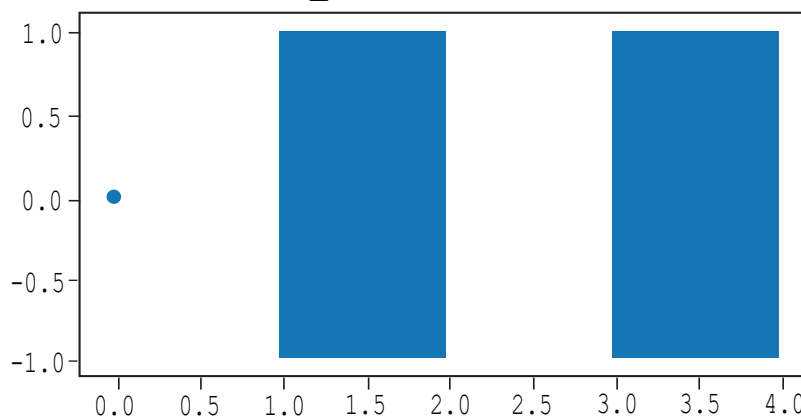


Figura 10. Función `envelope`.

Observa que al aplicar la función `envelope` a un punto, el resultado es el punto mismo, en el caso de la `LineString` y el polígono, podemos observar como la función nos regresa el rectángulo más pequeño que cubre totalmente la figura.

`GeoSeries.simplify(tolerance, preserve_topology=True)`

Este método nos regresa una `GeoSeries` que contiene una **versión simplificada de cada objeto**.

En el ejemplo, podemos ver que la versión simplificada de un punto es el punto mismo, la versión simplificada de una `LineString` es una simple línea que tiene como extremos los límites o `boundaries` de la `LineString` original, y en el caso del polígono, el resultado es el polígono mismo.

```
s.simplify(1).plot()
```

<matplotlib.axes._subplots.AxesSubplot at 0x7fa06b669110>

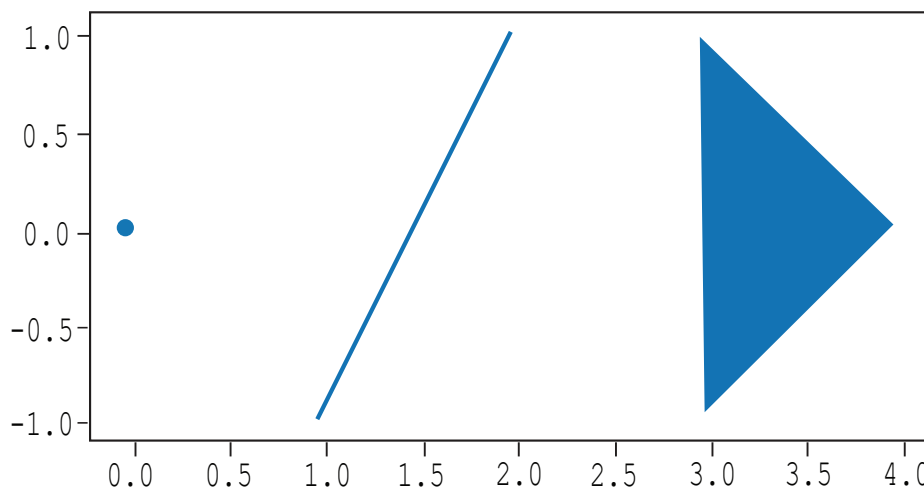


Figura 11. Función `simplify`.

GeoSeries.unary_union

Este método nos regresa una geometría que contiene la **unión de todas las geometrías contenidas en la GeoSeries**. Para ver cómo quedaría esto con nuestro mismo ejemplo, utilizamos la instrucción:

```
s.unary_union
```

y el resultado es el siguiente:

```
GEOMETRYCOLLECTION (POINT (0 0), LINESTRING (1 -1, 1 0, 2 0, 2 1), POLYGON ((3 -1, 4 0, 3 1, 3 -1)))
```

s.unary_union

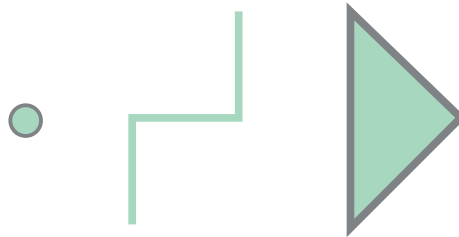


Figura 12. Función unary_union.

Antes de continuar...

Acabamos de revisar algunas funciones de las `GeoSeries` que pueden ayudarnos en distintas situaciones, por ejemplo:

- Encontrar un punto representativo de una región.
- Definir áreas poco extendidas en una geometría.
- Identificar límites.
- Crear polígonos que encierran ciertas regiones.
- Generar rectángulos envolventes que cubren por completo una geometría.
- Obtener versiones simplificadas de geometrías y fusionar múltiples geometrías en una sola.

En tu día a día, ¿puedes pensar en algún tipo de información en la que puedas utilizar algunas de las funciones revisadas?

Durante la siguiente semana, analiza el espacio físico de tu empresa y trata de visualizar:

- ¿Qué regiones están contenidas dentro de otras más grandes?
- ¿Te serviría delimitar las regiones para distinguirlas mejor?
- ¿Cómo usarías los puntos representativos para mejorar la visualización, por ejemplo, de zonas seguras o puntos de reunión durante un temblor?

| Transformaciones

En esta sección, nos adentraremos en los **transformadores**, un conjunto de métodos poderosos que nos permiten realizar cambios sobre las geometrías contenidas en una `GeoSeries`. A través de funciones como `rotate`, `scale`, `skew` y `translate`, exploraremos cómo transformar las geometrías en distintos ejes y con diferentes puntos de referencia, asegurando que los objetos geométricos se adapten de manera precisa a los requerimientos del proyecto.

■ `GeoSeries.affine_transform(self, matrix)`

Transforma las geometrías de la `GeoSeries` **utilizando una matriz de transformación**. La matriz de transformación debe ir expresada en forma de una tupla de 6 o 12 elementos, dependiendo de si es una transformación 2D o 3D.

Por ejemplo, para una transformación 2D, la matriz es:

$$[a, b, d, e, x_{\text{off}}, y_{\text{off}}]$$

que representa la matriz de transformación aumentada:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & x_{\text{off}} \\ d & e & y_{\text{off}} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

o visto de otra forma:

$$\begin{aligned}x' &= ax + by + x_{\text{off}} \\ y' &= dx + ey + y_{\text{off}}.\end{aligned}$$

donde x' y y' son las coordenadas del punto transformado.

De manera similar podemos trabajar con coordenadas 3D.

Esta función de transformación es la más general. Las funciones que veremos a continuación son formas más sencillas de aplicar transformaciones a geometrías, pero todas pueden expresarse como una transformación.

■ `GeoSeries.rotate(self, angle, origin='center', use_radians=False)`

Esta función **rota las coordenadas de las GeoSeries** utilizando una matriz de rotación 2D con un ángulo θ .

$$\begin{bmatrix} \cos \theta & -\sin \theta & x_{\text{off}} \\ \sin \theta & \cos \theta & y_{\text{off}} \\ 0 & 0 & 1 \end{bmatrix}$$

donde los desplazamientos se calculan a partir del origen de coordenadas.

$$\begin{aligned}x_{\text{off}} &= x_0 - x_0 \cos \theta + y_0 \sin \theta \\ y_{\text{off}} &= y_0 - x_0 \sin \theta - y_0 \cos \theta\end{aligned}$$

La función `rotate` **recibe como parámetro el ángulo** (por *default* en grados).

Opcionalmente, se puede especificar el origen como eje de rotación y elegir trabajar con el ángulo en radianes.

Veamos un ejemplo.

Nuevamente, definamos una `GeoSeries` con 3 elementos, un punto, una `LineString`, y un polígono:

```
s1 = geopandas.GeoSeries(  
    [  
        Point(1, 1),  
        LineString([(2, -1), (1, -1), (1, 0)]),  
        Polygon([(3, -1), (4, 0), (3, 1)]),  
    ]  
)
```

```
s1.plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f7cea63cc10>
```

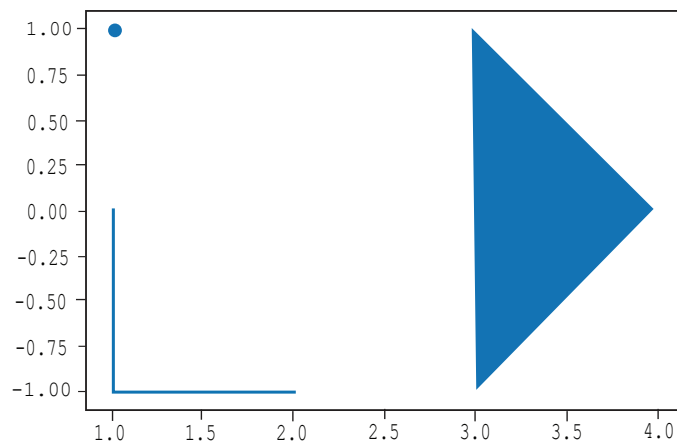


Figura 13. Definición de una `GeoSeries` con 3 elementos.

Ahora, apliquemos una rotación usando `.rotate` por ejemplo, de 90 grados con respecto al origen de los ejes coordenados:

```
s1.rotate(90, origin=(0,0) ).plot()
```

```
s1.rotate(90, origin=(0,0)).plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f7cea6d9590>
```

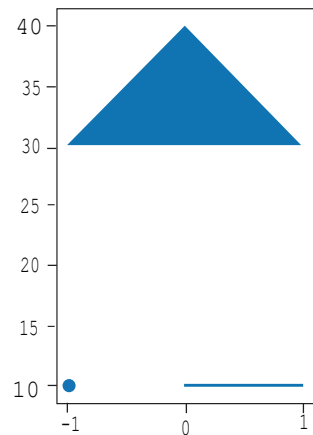


Figura 14. Función `rotate` aplicada.

Observa que se rotó 90 grados con respecto al origen de los ejes coordenados.

A menudo, en lugar de rotar con respecto al origen de coordenadas, puede ser más útil rotar cada geometría en torno a su centroide para mantener su posición relativa con otras geometrías. Por ejemplo, podemos utilizar:

```
s1.rotate(90, origin='centroid').plot()
```

```
s1.rotate(90, origin=('centroid')).plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f7cea6c1610>
```

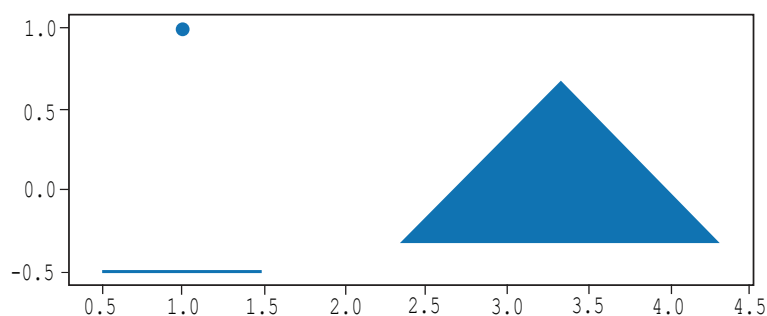


Figura 15. Función `rotate` en torno al centroide.

De esta forma, las geometrías conservan su posición relativa a las demás, pero rotan con respecto a su centroide en los grados indicados.

```
GeoSeries.scale(self, xfact=1.0, yfact=1.0, zfact=1.0,
origin='center')
```

Esta operación **escala las geometrías de la GeoSerie** a lo largo de cada dimensión (**X, Y**). Continuemos con el ejemplo del punto, la `LineString` y el polígono:

```
s1 = geopandas.GeoSeries(
[
    Point(1, 1),
    LineString([(2, -1), (1, -1), (1, 0)]),
    Polygon([(3, -1), (4, 0), (3, 1)]),
])
```

```
s1.plot()
```

<matplotlib.axes._subplots.AxesSubplot at 0x7f7cea63cc10>

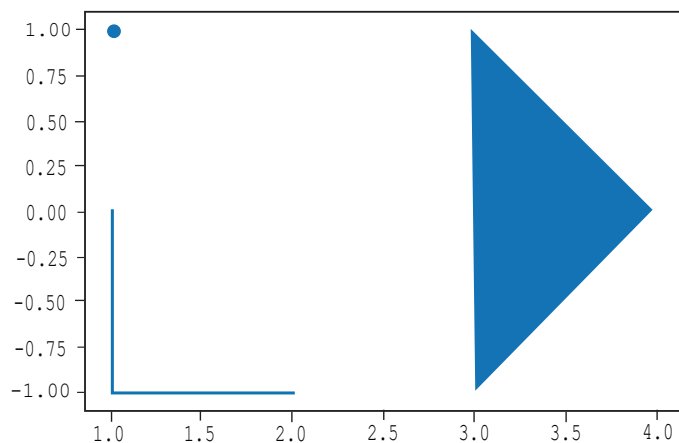


Figura 15. Gráfica con el punto, la `LineString` y el polígono.

Hagamos un escalado, por ejemplo, de 1.2 en x , y de 1.5 en y con respecto al origen:

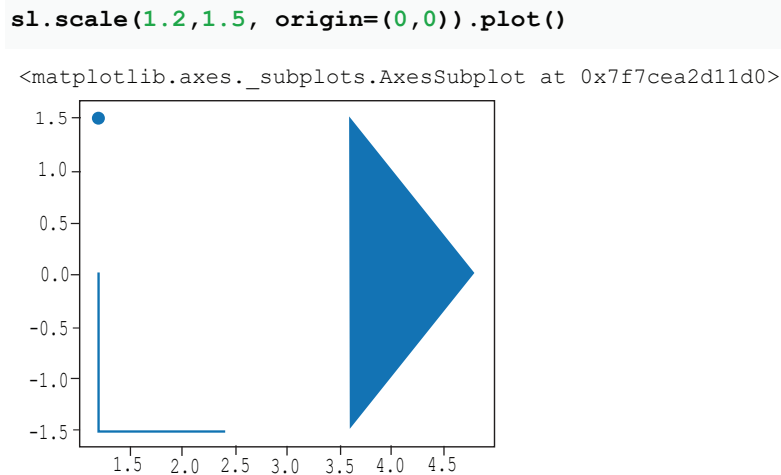


Figura 16. Escalado de los elementos.

Veamos las diferencias entre la gráfica original y la gráfica escalada. En la GeoSeries original, el lado horizontal de la `LineString` iba de 1.0 a 2.0 en el eje x , con una longitud de una unidad. Tras la transformación, ahora comienza en 1.2 y termina en 2.4, resultando en una longitud de 1.2 unidades. De manera similar, podríamos analizar todas las demás geometrías.

Realicemos otro ejemplo, haciendo el escalado a partir del centroide de las geometrías:

```
s1.scale(1.2,1.5, origin='centroid').plot()
```

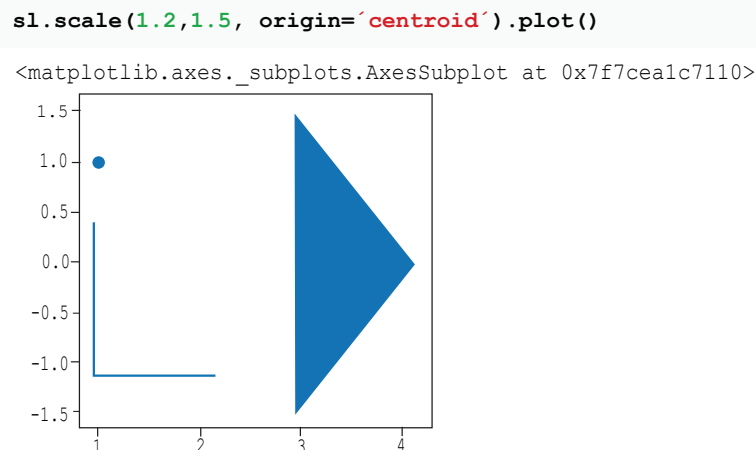


Figura 17. Elementos escalados a partir del centroide.

```
ss.skew(self, angle, origin='center', use_radians=False)
```

Esta transformación inclina las geometrías de las GeoSeries de acuerdo con ángulos en las dimensiones ***X*** y ***Y***. La mejor forma de entender esta transformación es con un ejemplo:

Veamos primero una transformación `skew` con 30 grados en ***x***, y 0 en ***y***

```
s1.skew(30, 0).plot()
```

```
s1.skew(30, 0).plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f7cea0ef410>
```

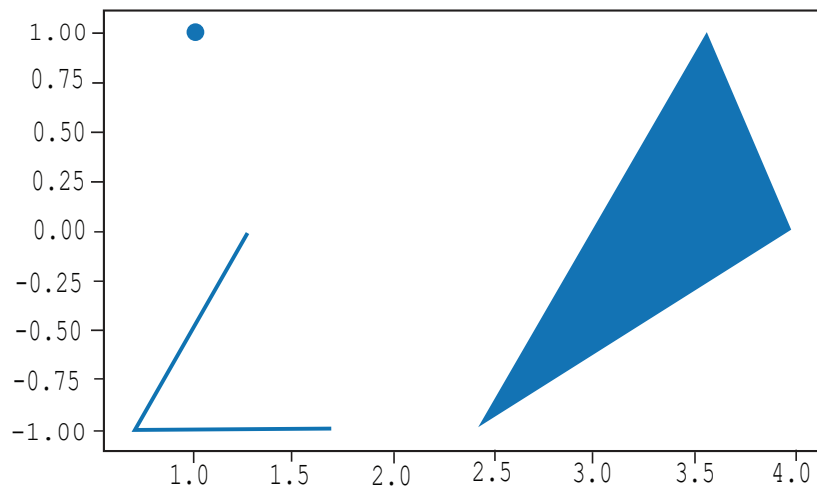


Figura 18. Función `skew`.

Ahora pongamos 0 grados en x y 15 grados en y :

```
s1.skew(0, 15).plot()
```

```
s1.skew(0, 15).plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f7ce9fc76d0>
```

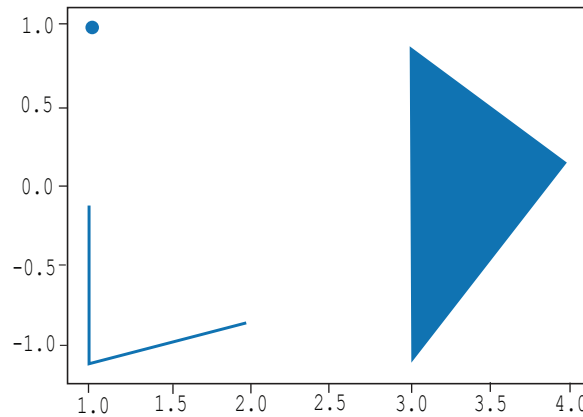


Figura 19. Función `skew` con 0 grados en x y 15 grados en y .

Como podemos observar, esta transformación inclina las geometrías en un eje determinado o, si se desea, en ambos simultáneamente. Opcionalmente, el ángulo puede expresarse en radianes agregando el parámetro `use_radians=True`.

```
s1.skew(30, 15).plot()
```

```
s1.skew(30, 15).plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f7ce9f11f10>
```

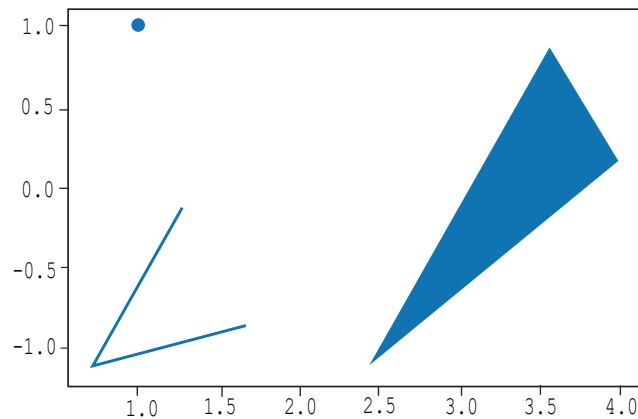


Figura 20. Función `skew` con el parámetro `use_radians=True`.

```
GeoSeries.translate(self, xoff=0.0, yoff=0.0, zoff=0.0)
```

Esta transformación **desplaza las coordenadas de las GeoSeries**, es decir, las mueve en un cierto eje. Esta transformación puede ser la más fácil de entender, pues recibe parámetros del desplazamiento en el eje **X**, en el eje **Y**, (y en el eje **Z** si estamos trabajando en 3D).

Un desplazamiento positivo en el eje **X** significa mover el objeto a la derecha, un desplazamiento negativo significa moverlo a la izquierda. De la misma manera, en el eje **Y**: negativo es hacia abajo, positivo hacia arriba.

Continuando con nuestra `GeoSeries` de las 3 geometrías, si aplicamos un desplazamiento de una unidad en el eje **X**:

```
s1.translate(1,0).plot()
```

```
s1.translate(1,0).plot()
```

<matplotlib.axes._subplots.AxesSubplot at 0x7f7ce9e38710>

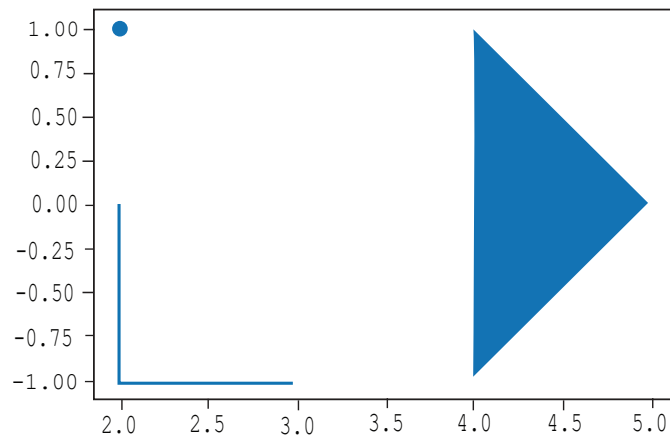


Figura 21. Transformación `translate`.

Podemos ver que los gráficos son muy parecidos, con la diferencia de que el eje **X** tiene los valores una unidad mayor.

También podemos hacer traslaciones sólo en el eje **Y**, y en ambos simultáneamente, como se muestra a continuación:

```
s1.translate(3,-2).plot()
```

```
s1.translate(3,-2).plot()
```

<matplotlib.axes._subplots.AxesSubplot at 0x7f7ce9ebf790>

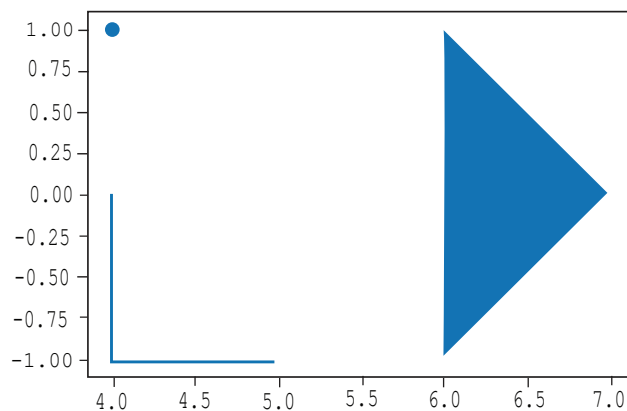


Figura 22. Función `translate` aplicada simultáneamente en eje **Y**.

Antes de terminar...

En este subtema revisamos algunas transformaciones que podemos aplicar a las geometrías de una `GeoSeries`, transformaciones como **rotación**, **escalado**, **inclinación** y **traslación**. Vimos algunos ejemplos de cómo aplicar estas funciones en Python.

Ahora el turno es tuyo, puedes pensar en tu lugar de trabajo, asumiendo que ya tienes identificada información geográfica y que ya la has graficado. ¿Puedes pensar en qué situaciones podrías aplicar algunas o todas estas transformaciones?

Glosario

- **Affine Transform:** Es una operación que transforma las geometrías mediante una matriz de transformación. Este tipo de transformación mantiene las proporciones y los ángulos de las figuras, pero puede cambiar su escala, rotación y posición. Las transformaciones afines se expresan con matrices de 2D o 3D.
- **Boundary:** Es el conjunto de límites que definen el contorno de una geometría. Puede representar los puntos extremos o el perímetro que los circunda.
- **Buffer:** Es un área que se genera alrededor de una geometría, dentro de una distancia específica. Se utiliza para identificar las áreas cercanas a un objeto y puede ser útil para realizar análisis de proximidad.
- **Centroide:** Es el punto que representa el centro geométrico de una figura. Es el “promedio” de todas las coordenadas dentro de la geometría, y puede ser útil para ubicar el punto central de una región o figura.
- **Convex Hull o casco convexo:** Es el polígono más pequeño que puede rodear completamente un conjunto de puntos. Es un polígono convexo cuyos ángulos internos no superan los 180 grados y es útil para encontrar el contorno más estrecho que cubre una forma compleja.
- **Envelope:** Es el polígono rectangular más pequeño que puede contener una geometría, con los lados paralelos a los ejes coordenados. Se utiliza para generar un límite simple y recto que encierra completamente una figura o conjunto de puntos.
- **Rotate:** es una transformación que rota una geometría alrededor de un punto específico (por defecto, el origen de coordenadas o el centroide) utilizando un ángulo determinado. Se utiliza para cambiar la orientación de las figuras en un espacio determinado.

- **Scale:** Ajusta el tamaño de una geometría a lo largo de los ejes **X**, **Y** y **Z**, multiplicando las coordenadas por factores de escala definidos. Puede utilizarse para ampliar o reducir una geometría, ya sea de forma uniforme o en direcciones específicas, con respecto a un punto de referencia como el centroide.
- **Simplify:** Se utiliza para reducir la complejidad de una geometría, eliminando detalles innecesarios y creando una versión más simple pero que mantiene las características fundamentales. Es útil para mejorar el rendimiento y la visualización de grandes conjuntos de datos.
- **Skew:** Es una transformación que inclina las geometrías en las dimensiones **X** y **Y**, de acuerdo con un ángulo determinado. Esta operación distorsiona las figuras, cambiando su forma de manera no uniforme, y puede ser útil para simular efectos como la perspectiva o para ajustar la geometría de acuerdo con diferentes necesidades.
- **Translate:** Es una transformación que desplaza las geometrías a lo largo de los ejes **X**, **Y** (y **Z**, en 3D). Permite mover las figuras a una nueva ubicación en el espacio, especificando los desplazamientos en cada uno de los ejes. Es útil para ajustar la posición de las geometrías sin cambiar su forma.
- **Unary Union:** Es una operación que combina todas las geometrías contenidas en una **GeoSeries** en una única geometría. Esta operación es útil para fusionar múltiples objetos espaciales en uno solo, simplificando el análisis de grandes volúmenes de datos.

Bibliografía

GeoPandas. (s.f.). *geopandas.GeoSeries.affine_transform*. Geopandas. Recuperado el 08 de abril 2025 de: https://geopandas.org/en/latest/docs/reference/api/geopandas.GeoSeries.affine_transform.html

GeoPandas. (s.f.). *geopandas.GeoSeries.boundary*. Geopandas. Recuperado el 08 de abril 2025 de: <https://geopandas.readthedocs.io/en/latest/docs/reference/api/geopandas.GeoSeries.boundary.html>

GeoPandas. (s.f.). *geopandas.GeoSeries.buffer*. Geopandas. Recuperado el 08 de abril 2025 de: <https://geopandas.readthedocs.io/en/latest/docs/reference/api/geopandas.GeoSeries.buffer.html>

GeoPandas. (s.f.). *geopandas.GeoSeries.centroid*. Geopandas. Recuperado el 08 de abril 2025 de: <https://geopandas.org/en/latest/docs/reference/api/geopandas.GeoSeries.centroid.html>

GeoPandas. (s.f.). *geopandas.GeoSeries.convex_hull*. Geopandas. Recuperado el 08 de abril 2025 de: https://geopandas.org/en/latest/docs/reference/api/geopandas.GeoSeries.convex_hull.html

GeoPandas. (s.f.). *geopandas.GeoSeries.envelope*. Geopandas. Recuperado el 08 de abril 2025 de: <https://geopandas.org/en/latest/docs/reference/api/geopandas.GeoSeries.envelope.html>

GeoPandas. (s.f.). *geopandas.GeoSeries.skew*. Geopandas. Recuperado el 08 de abril 2025 de: <https://geopandas.org/en/latest/docs/reference/api/geopandas.GeoSeries.skew.html>

GeoPandas. (s.f.). *geopandas.GeoSeries.rotate*. Recuperado el 08 de abril 2025 de: <https://geopandas.org/en/latest/docs/reference/api/geopandas.GeoSeries.rotate.html>

GeoPandas. (s.f.). *geopandas.GeoSeries.scale*. Recuperado el 08 de abril 2025 de: <https://geopandas.org/en/latest/docs/reference/api/geopandas.GeoSeries.scale.html>

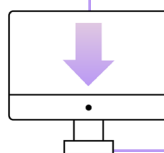
GeoPandas. (s.f.). *geopandas.GeoSeries.simplify*. Geopandas. Recuperado el 08 de abril 2025 de: <https://geopandas.org/en/latest/docs/reference/api/geopandas.GeoSeries.simplify.html>

GeoPandas. (s.f.). *geopandas.GeoSeries.translate*. Geopandas. Recuperado el 08 de abril 2025 de: <https://geopandas.org/en/latest/docs/reference/api/>

GeoPandas. (s.f.). *geopandas.GeoSeries.unary_union*. Geopandas. Recuperado el 08 de abril 2025 de: https://geopandas.org/en/v0.10.0/docs/reference/api/geopandas.GeoSeries.unary_union.html

Shapely. (s.f.). *Affine Transformations*. Shapely 2.1.0. Recuperado el 08 de abril 2025 de: https://shapely.readthedocs.io/en/stable/manual.html#shapely.affinity.affine_transform

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 08 de abril del 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.